VQC auf Iris-Datensatz - AerSimulator

Modell: Variational Quantum Classifier (VQC)
Datensatz: Iris (3 Klassen, 2 Features)
Backend: AerSimulator (lokal)
Ansatz: RealAmplitudes
Feature Map: zz_feature_map (2 Qubits)

Zweites Quantenmodell nach Quantum Kernel SVM. Direkter Vergleich möglich da gleiche Feature Map, gleicher Datensatz-Split.

In [3]:
```
import sys
import os
sys.path.append(os.path.abspath(".."))  # utils.py im Root

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report

from qiskit.circuit.library import real_amplitudes, zz_feature_map
from qiskit_machine_learning.algorithms import VQC
from qiskit.primitives import StatevectorSampler
from qiskit_aer.primitives import Sampler as AerSampler
#from qiskit_machine_learning.optimizers import COBYLA
from qiskit_machine_learning.optimizers import SPSA

from utils import save_result

import warnings
warnings.filterwarnings("ignore")

print("Imports OK")
```

```
Imports OK
```

1. Datensatz laden

Gleicher Split wie Quantum Kernel Experiment: 2 Features (Index 0, 2), random_state=42, test_size=0.3 - für Vergleichbarkeit.

In [14]:
```
iris = load_iris()
X = iris.data[:, [0, 2]]  # sepal length, petal length - 2 Features → 2 Qubi
y = iris.target            # 3 Klassen: 0, 1, 2

# Train/Test Split (gleich wie Quantum Kernel)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Normalisierung auf [0, 2π] - für Winkelkodierung
scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

print(f"Training:  {X_train_scaled.shape}")
print(f"Test:      {X_test_scaled.shape}")
print(f"Klassen:   {np.unique(y_train)}")
```

```
Training:  (105, 2)
Test:      (45, 2)
Klassen:   [0 1 2]
```

2. Quantum Circuit - Feature Map + Ansatz

- **Feature Map:** `zz_feature_map`, 2 Qubits, 1 Repetition
- **Ansatz:** `real_amplitudes`, 2 Qubits, 2 Repetitionen
- Kombination ergibt den trainierbaren VQC-Circuit

In [15]:
```
num_qubits = 2

# Feature Map (Datenkodierung)
feature_map = zz_feature_map(feature_dimension=num_qubits, reps=1)

# Ansatz (trainierbare Parameter)
ansatz = real_amplitudes(num_qubits=num_qubits, reps=2)

print("Feature Map:")
print(feature_map.decompose())
print()
print("Ansatz:")
print(ansatz)
print()
print(f"Trainierbare Parameter: {ansatz.num_parameters}")
```

```
Feature Map:
                                                                                »
0: ┤ U(π/2,0,π) ├┤ U(0,0,2*x[0]) ├──■──────────────────────────────────────────»
     ├─────────┤├──────────────┤┌─┴─┐┌──────────────────────────────────────┐»
1: ┤ U(π/2,0,π) ├┤ U(0,0,2*x[1]) ├┤ X ├┤ U(0,0,(-π + x[0])*(-π + x[1])*2) ├»
     └─────────┘└──────────────┘└───┘└──────────────────────────────────────┘»
«
«0: ──■──
«    ┌─┴─┐
«1: ┤ X ├
«    └───┘

Ansatz:
     ┌──────────┐     ┌──────────┐     ┌──────────┐
q_0: ┤ Ry(θ[0]) ├──■──┤ Ry(θ[2]) ├──■──┤ Ry(θ[4]) ├
     ├──────────┤┌─┴─┐├──────────┤┌─┴─┐├──────────┤
q_1: ┤ Ry(θ[1]) ├┤ X ├┤ Ry(θ[3]) ├┤ X ├┤ Ry(θ[5]) ├
     └──────────┘└───┘└──────────┘└───┘└──────────┘

Trainierbare Parameter: 6
```

3. VQC Training

COBYLA als Optimizer - gradientenfrei, robust für NISQ.
`max_iter=150` als Kompromiss zwischen Konvergenz und Laufzeit.
Der `AerSampler` führt die Messungen auf dem lokalen Simulator aus.

In [30]:
```python
import time

# Sampler für AerSimulator
sampler = StatevectorSampler()

# Optimizer
optimizer = SPSA(maxiter=150)

# VQC Instanz
vqc = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=optimizer,
    sampler=sampler,
)

# Training
print("Starte Training...")
start = time.time()
vqc.fit(X_train_scaled, y_train)
train_time = round(time.time() - start, 2)

print(f"Training abgeschlossen in {train_time}s")
```

```
No gradient function provided, creating a gradient function. If your Sampler
requires transpilation, please provide a pass manager.
```

```
Starte Training...
Training abgeschlossen in 263.48s
```

4. Evaluation

In [31]:
```python
start = time.time()

y_pred = vqc.predict(X_test_scaled)

infer_time = round(time.time() - start, 2)

accuracy = accuracy_score(y_test, y_pred)
f1       = f1_score(y_test, y_pred, average="weighted")

print(f"Accuracy: {accuracy:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Trainingszeit:  {train_time}s")
print(f"Inferenzzeit:   {infer_time}s")
print()
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
Accuracy: 0.5111
F1-Score: 0.4099
Trainingszeit:  263.48s
Inferenzzeit:   0.38s

Classification Report:
              precision    recall  f1-score   support

      setosa       0.59      0.67      0.62        15
  versicolor       0.46      0.87      0.60        15
   virginica       0.00      0.00      0.00        15

    accuracy                           0.51        45
   macro avg       0.35      0.51      0.41        45
weighted avg       0.35      0.51      0.41        45
```

5. Ergebnis speichern

In [32]:
```python
save_result(
    modell="VQC (RealAmplitudes); Optimizer: SPSA; 150 Iterations",
    datensatz="Iris (3 Klassen, 2 Features)",
    backend="AerSimulator",
    accuracy=round(accuracy, 4),
    f1=round(f1, 4),
    trainingszeit=train_time,
    inferenzzeit=infer_time,
)

print("Gespeichert in ergebnisse.csv")
```

```
                                                  Modell                    Dat
ensatz      Backend  Accuracy     F1  Trainingszeit_s  Inferenzzeit_s  Featu
re Map  Reps
                                  Quantum Kernel SVM Iris (2 Klassen, 2 Fea
tures) AerSimulator    0.6500 0.5882          16.8214          8.9400 ZZFeat
ureMap   2.0
                                Klassischer SVM (RBF) Iris (2 Klassen, 2 Fea
tures)        lokal    1.0000 1.0000           0.0037          0.0008
NaN   NaN
                                  Quantum Kernel SVM Iris (3 Klassen, 2 Fea
tures) AerSimulator    0.6667 0.6453          37.3539         18.7609 ZZFeat
ureMap   2.0
                                Klassischer SVM (RBF) Iris (3 Klassen, 2 Fea
tures)        lokal    0.9000 0.8992           0.0045          0.0011
NaN   NaN
                                             MLP Iris (3 Klassen, 2 Fea
tures)        lokal    0.8333 0.8321           0.9989          0.0012     (6
4, 32)   NaN
                                 VQC (RealAmplitudes) Iris (3 Klassen, 2 Fea
tures) AerSimulator    0.4000 0.3196          39.0800          0.3400
NaN   NaN
    VQC (RealAmplitudes); mehr Interationen 2x -> 300 Iris (3 Klassen, 2 Fea
tures) AerSimulator    0.3111 0.2476          44.9100          0.3500
NaN   NaN
VQC (RealAmplitudes); Optimizer: SPSA; 300 Iterations Iris (3 Klassen, 2 Fea
tures) AerSimulator    0.5333 0.4222         489.5500          0.3700
NaN   NaN
VQC (RealAmplitudes); Optimizer: SPSA; 150 Iterations Iris (3 Klassen, 2 Fea
tures) AerSimulator    0.5111 0.4099         263.4800          0.3800
NaN   NaN
Gespeichert in ergebnisse.csv
```

6. Aktueller Vergleich (Iris)

Kurzer Blick auf den Stand der ergebnisse.csv für Iris.

In [4]:
```
df = pd.read_csv("Ergebnisse/ergebnisse.csv")

# Nur Iris, nur 3-Klassen
df_iris = df[df["Datensatz"].str.contains("3 Klassen")]
print(df_iris[["Modell", "Backend", "Accuracy", "F1"]].to_string(index=False
```

```
                             Modell            Backend Accuracy     F1
                Quantum Kernel SVM       AerSimulator   0.6222  0,616
              VQC (RealAmplitudes)       AerSimulator   0.5333 0.4222
             Klassischer SVM (RBF)              lokal   0.9556 0.9556
                               MLP              lokal   0.9333 0.9327
       Quantum Kernel SVM (Noise) AerSimulator+Noise   0.6222  0.616
     VQC (RealAmplitudes, Noise) AerSimulator+Noise   0.5778 0.4617
    VQC (RealAmplitudes, COBYLA)       AerSimulator   COBYLA    100
VQC (RealAmplitudes, COBYLA, Noise) AerSimulator+Noise   COBYLA    100
```